

Basic C Without Tears

Guía Práctica con Soluciones – Tipo A – Guía 02

Fundamentos de C, traza de código, strings y matrices

Autor: Stack

1. **Fundamentos del Lenguaje (Teoría aplicada).** Para los cálculos de tamaño, suponga que un `char` ocupa 1 byte, un `int` 4 bytes y un `double` 8 bytes. Suponga también que `unsigned char` tiene 8 bits.

- (a) Calcule el tamaño de cada objeto declarado y la suma de esos tamaños, en bytes. No considere posibles espacios entre variables.

```
char etiqueta[6];
int lecturas[3];
double promedio;
```

Solución y Explicación:

`etiqueta` ocupa $6 \times 1 = 6$ bytes; `lecturas` ocupa $3 \times 4 = 12$ bytes; `promedio` ocupa 8 bytes. En total son $6 + 12 + 8 = 26$ bytes.

- (b) Determine si la siguiente expresión es verdadera o falsa en C tradicional e indique su valor entero. Justifique cómo se interpreta el operando negativo.

```
(0 || -2) && (5 % 2)
```

Solución y Explicación:

`0 || -2` vale 1 porque `-2` es distinto de cero. `5 % 2` vale 1. Por tanto, `1 && 1` es verdadero y su resultado entero es 1. En una condición, cero es falso y cualquier valor distinto de cero es verdadero; los operadores lógicos producen exactamente 0 o 1.

- (c) Si `dato` es un `unsigned char` de 8 bits, ¿qué valor queda almacenado después de la asignación? Explique el resultado.

```
1 unsigned char dato = 255;
2 dato = dato + 1;
```

Solución y Explicación:

El valor almacenado es **0**. Para la suma, `dato` se promueve a `int`, por lo que se calcula $255 + 1 = 256$. Al asignarlo a `unsigned char`, se reduce módulo 256: el resto es 0. Esta conversión sin signo está definida; no ocurre desbordamiento de `int`.

- 2. Análisis y Traza de Código (Matemáticas y Control de Flujo).** ¿Qué imprime exactamente este código en pantalla? Suponga que el fragmento está dentro de `main` y que se incluyó `stdio.h`. Sin ejecutarlo ni modificarlo, escriba la salida respetando espacios y saltos de línea. Muestre los valores de `i` y `total` al terminar cada iteración.

```
1 int i = 1;
2 int total = 0;
3 while (i <= 4) {
4     total += i++ % 3;
5     if (i % 2 == 0) {
6         printf("P%d ", total);
7     } else {
8         printf("I%d ", i);
9     }
10 }
11 switch (total % 3) {
12     case 1: printf("uno ");
13     case 2: printf("dos ");
14     default: printf("fin\n");
15 }
```

Solución y Explicación:

En cada iteración, `i++` aporta el valor anterior de `i` a la operación y luego incrementa `i`. Los estados y textos son:

i inicial	total final	i final	Impresión
1	1	2	P1
2	3	3	I3
3	3	4	P3
4	4	5	I5

Al terminar el ciclo, `total % 3` vale 1. El `switch` entra en `case 1`; como no hay `break`, continúa por `case 2` y luego por `default`.

Salida exacta:

P1 I3 P3 I5 uno dos fin

Hay un espacio entre cada texto y un único salto de línea después de `fin`.

- 3. Algoritmia con Strings (Pseudocódigo).** Escriba el pseudocódigo de una función que elimine de una cadena `texto` todas las apariciones del carácter `objetivo`, modificando la misma cadena. Suponga que `objetivo` es distinto de `'\0'`. Recorra los caracteres hasta encontrar ese terminador, conserve el orden de los caracteres restantes y coloque correctamente el terminador al finalizar. No utilice funciones de biblioteca ni un arreglo auxiliar. Describa qué ocurre si la cadena está vacía, si el carácter no aparece o si todos los caracteres coinciden con él.

Texto	Objetivo	Resultado
"banana"	'a'	"bnn"
"sol"	'x'	"sol"
"aaa"	'a'	""
""	'a'	""

Solución y Explicación:

```
FUNCION eliminar_caracter(texto, objetivo)
  lectura <- 0
  escritura <- 0

  MIENTRAS texto[lectura] != '\0' HACER
    SI texto[lectura] != objetivo ENTONCES
      texto[escritura] <- texto[lectura]
      escritura <- escritura + 1
    FIN_SI
    lectura <- lectura + 1
  FIN_MIENTRAS

  texto[escritura] <- '\0'
FIN_FUNCION
```

`lectura` visita todos los caracteres y `escritura` solo avanza cuando se conserva uno. Como la escritura nunca adelanta a la lectura, no se sobrescriben caracteres pendientes. El orden se conserva.

Si la cadena está vacía o todos los caracteres se eliminan, se escribe el terminador en la posición 0. Si el objetivo no aparece, la cadena queda igual.

4. **Algoritmia con Matrices/Arreglos (Pseudocódigo).** Una matriz `notas` tiene `filas` filas y `columnas` columnas, ambas cantidades positivas. Escriba pseudocódigo con bucles anidados e índices `i` y `j`, comenzando en cero, que calcule e informe el promedio de cada fila. La división debe ser real, sin descartar decimales. Para comprobar la solución, use:

$$\text{notas} = \begin{bmatrix} 4 & 8 & 6 & 2 \\ 3 & 5 & 7 & 9 \\ 10 & 0 & 2 & 5 \end{bmatrix}$$

Indique los tres promedios obtenidos.

Solución y Explicación:

```
PARA i <- 0 HASTA filas - 1 HACER
    suma <- 0.0
    PARA j <- 0 HASTA columnas - 1 HACER
        suma <- suma + notas[i][j]
    FIN_PARA
    promedio <- suma / columnas // division real
    ESCRIBIR "Promedio de fila ", i, ": ", promedio
FIN_PARA
```

Se reinicia `suma` en cada fila; de lo contrario se acumularían notas de filas anteriores. Los límites incluyen todos los elementos sin salir de la matriz.

- Fila 0: $(4 + 8 + 6 + 2)/4 = 5$.
- Fila 1: $(3 + 5 + 7 + 9)/4 = 6$.
- Fila 2: $(10 + 0 + 2 + 5)/4 = 4,25$.